

RansomShield — Aide-mémoire de soutenance

Discours complet — 10 min de présentation + 5 min de démo

Codjo Fréjus Loïc SANGRONIO — 24 juin 2026

À lire au calme avant la soutenance ; en séance, s'en servir comme filet de sécurité, pas comme texte à réciter mot à mot. Le numéro « Diapo N » correspond exactement au numéro affiché dans le panneau de diapositives d'Impress : si vous êtes sur la bonne note, vous êtes sur la bonne diapo. Les **flèches vertes** sont des phrases à prononcer pour enchaîner naturellement vers la diapo suivante.

Diapo 1/24 — OUVERTURE — 0 :00

Excellence Monsieur le Président du jury, Mesdames et Messieurs les membres du jury, bonjour. Permettez-nous de vous exprimer notre profonde gratitude pour avoir bien voulu accepter d'évaluer ce travail malgré vos agendas sûrement chargés. Le sujet ayant fait l'objet de notre étude est intitulé : *Conception et mise en place d'un système de détection et de réponse aux attaques ransomware dans un environnement contrôlé* : RansomShield. Cette présentation se déroulera en dix minutes, suivie d'une démonstration live de cinq minutes sur des machines virtuelles réelles. Nous resterons à votre entière disposition pour vos questions à l'issue de cet exposé.

↪ **Voici comment s'articule notre présentation.**

Diapo 2/24 — SOMMAIRE — 0 :30

Notre présentation s'articule en cinq grandes parties. Nous aborderons dans un premier temps le contexte et la problématique qui ont motivé ce travail. Nous passerons ensuite en revue la littérature existante sur le sujet. Nous présenterons par la suite la conception et la réalisation du système. Nous exposerons les résultats obtenus avant de conclure par une synthèse et les perspectives d'évolution. Le tout sera couronné d'une démonstration du système en conditions réelles.

↪ **Commençons par le contexte qui a motivé notre démarche.**

Diapo 4/24 — CONTEXTE — 1 :00

Nous vivons à une époque où les organisations dépendent fortement de leurs systèmes informatiques. Cette dépendance les expose à une menace croissante : le ransomware. Il s'agit d'un logiciel malveillant qui chiffre les données d'une organisation, puis réclame une rançon pour en rétablir l'accès. L'ENISA a enregistré une hausse de 73 % de ces attaques en 2023 ; sans outil dédié, il faut en moyenne 21 jours pour détecter un tel incident, pour un coût qui dépasse souvent le million de dollars. Les cibles ne se limitent plus aux grandes entreprises : les hôpitaux, les PME et les universités sont désormais tout aussi exposés.

↪ **Ce constat soulève une problématique précise.**

Diapo 5/24 — PROBLÉMATIQUE — 1 :45

Le problème fondamental réside dans les limites des outils traditionnels : les antivirus classiques détectent les malwares par empreinte connue et se révèlent inefficaces face aux nouvelles variantes. Trois défis structurent ce constat. **Premier défi** : les variantes inconnues. Nous avons besoin d'un système capable de détecter le comportement, et non la signature. **Deuxième défi** : le parc hétérogène. Une organisation réelle gère des machines sous Windows, Linux et macOS. L'agent doit

fonctionner sur toutes ces plateformes. **Troisième défi** : la réponse automatique non contrôlée est risquée. Isoler une machine sans vérification peut paralyser un service critique. Le contrôle humain n'est pas une contrainte — c'est une garantie. D'où la question centrale : comment détecter un comportement ransomware, dans un environnement contrôlé, sans déployer de vrai malware ?

↪ **C'est précisément ce que RansomShield cherche à réaliser.**

Diapo 6/24 — OBJECTIFS — 2 :30

Pour répondre à cette problématique, nous nous sommes fixé six objectifs spécifiques : surveiller les événements fichiers en temps réel ; analyser les comportements et calculer un score de risque configurable ; générer automatiquement des alertes et des incidents de sécurité ; proposer des actions de protection tout en garantissant un contrôle humain à chaque étape ; sécuriser la console par une double authentification ; valider l'ensemble sur des scénarios d'attaque représentatifs.

↪ **Voyons maintenant les fondements théoriques qui ont guidé notre démarche.**

Diapo 8/24 — DÉFINITION & COMPORTEMENTS — 3 :45

Le NIST définit le ransomware comme un logiciel malveillant qui bloque l'accès aux données — le plus souvent par chiffrement — pour ensuite réclamer une rançon à la victime. Ce qui présente un intérêt particulier pour notre travail, c'est que ces attaques ne sont pas silencieuses. Elles laissent des traces identifiables avant que les dégâts ne deviennent irréversibles : un renommage massif de fichiers avec une extension suspecte telle que `.locked`, la création d'une note de rançon, la suppression des points de restauration, ou encore le recours à des outils légitimes détournés — ce que l'on appelle les LOLBins. Ce sont précisément ces signaux comportementaux que notre moteur de détection surveille.

↪ **Mais pourquoi les outils existants ne permettent-ils pas de répondre à ce défi ?**

Diapo 9/24 — SOLUTIONS EXISTANTES — 4 :30

Les solutions du marché présentent chacune des limites significatives. Les antivirus classiques opèrent par reconnaissance d'empreintes connues et se révèlent aveugles aux nouvelles variantes. Les SIEM collectent des journaux d'activité sans offrir de corrélation comportementale ciblée. Quant aux EDR commerciaux — tels que CrowdStrike ou SentinelOne — ils sont efficaces mais inaccessibles pour la majorité des organisations : PME, hôpitaux, administrations, qui ne disposent pas des budgets requis. C'est dans cet espace que se positionne RansomShield : une approche comportementale, open-source, déployable sur toute infrastructure, avec un opérateur humain dans la boucle à chaque décision.

↪ **Voyons maintenant comment nous avons conçu ce système.**

Diapo 11/24 — ARCHITECTURE — 5 :15

L'architecture de RansomShield repose sur un flux simple que l'on peut lire de gauche à droite, en cinq étapes. Les agents Python, installés sur les machines surveillées, collectent les événements fichiers et les transmettent à l'API Laravel. Chaque agent dispose d'une clé qui lui est propre. L'API authentifie et achemine les données vers le moteur de détection, qui calcule un score de risque. Lorsque le seuil configuré est franchi, une alerte est créée et un incident s'ouvre automatiquement.

L'analyste SOC consulte alors la console et valide chaque action avant son exécution. Règle d'or : aucune action n'est exécutée sans validation humaine préalable.

→ **Détaillons les quatre composants qui constituent ce système.**

Diapo 12/24 — LES 4 COMPOSANTS — 6 :00

Notre système repose sur quatre composants complémentaires. L'**agent Python** surveille les fichiers en temps réel et conserve une file locale en SQLite pour ne perdre aucun événement, même en cas de coupure réseau. La **console SOC** centralise les alertes, les incidents, la file d'approbation et la timeline. Les notifications sont diffusées sur quatre canaux : web, e-mail, son et webhook. La **découverte réseau** détecte automatiquement les nouvelles machines, mais aucune ne peut rejoindre le système sans validation explicite de l'administrateur. Le **simulateur d'attaques** rejoue jusqu'à vingt-deux événements sans recourir à un vrai malware.

→ **Qui sont les acteurs qui interagissent avec ce système ?**

Diapo 13/24 — DIAGRAMME USE CASE — 6 :40

Notre système distingue deux acteurs aux rôles bien définis. L'administrateur assure la configuration du système, la gestion des agents et le lancement des simulations. L'analyste SOC se concentre quant à lui sur le traitement opérationnel : il visualise les alertes et les incidents, approuve ou rejette les actions de protection, et consulte la timeline d'audit. Il n'a cependant pas accès à la configuration. Cette séparation garantit qu'un analyste ne peut pas modifier les seuils de détection, et qu'un administrateur ne gère pas les incidents au quotidien.

→ **Comment les entités du système sont-elles structurées entre elles ?**

Diapo 14/24 — DIAGRAMME DE CLASSES — 7 :05

Notre modèle objet distingue sept entités principales. Le pipeline se lit de gauche à droite : un Agent génère des Events, que les DetectionRules évaluent. Lorsque le score dépasse le seuil, une Alert est créée, puis escaladée en Incident. Chaque Incident génère des ProtectionActions. L'ensemble est tracé dans un AuditLog.

→ **Comment ces entités se traduisent-elles en base de données ?**

Diapo 15/24 — MODÈLE DE DONNÉES — 7 :25

Notre modèle de données repose sur huit tables MySQL. Le pipeline se lit de gauche à droite : agent, événement, alerte, incident, action, journal d'audit. Chaque action est signée : on sait qui l'a validée, et à quel moment. Aucune décision n'est anonyme dans notre système.

→ **Passons maintenant à la réalisation concrète.**

Diapo 17/24 — CHOIX TECHNIQUES — 7 :55

Nos choix techniques ont été guidés par trois critères : la portabilité, la légèreté et la capacité à démontrer le système en conditions réelles. Le backend repose sur Laravel 11 avec PHP 8.3, une base de données MySQL et une API sécurisée par une clé propre à chaque agent. L'agent de surveillance

utilise Python avec Watchdog et une file locale SQLite. Le frontend intègre Chart.js localement, sans dépendance CDN externe. L'infrastructure de test est constituée de trois machines virtuelles KVM avec une authentification à double facteur.

↪ Voyons comment l'agent et le moteur de détection fonctionnent concrètement.

Diapo 18/24 — AGENT & MOTEUR — 8 :20

Au premier démarrage, l'agent s'inscrit auprès du serveur à l'aide d'un jeton à usage unique. Il obtient en retour une clé API permanente. Il débute ensuite la surveillance des dossiers configurés grâce à Watchdog et envoie un signal de présence toutes les trente secondes. Notre moteur de détection calcule un score cumulatif configurable. Exemple concret : un fichier renommé en `.locked` rapporte 80 points ; la création d'une note de rançon ajoute 55 points. Le total de 135 points franchit le seuil critique : une alerte et un incident sont créés automatiquement en moins de cinq secondes.

↪ Voyons ce que la console SOC offre à l'analyste.

Diapo 19/24 — CONSOLE SOC — 9 :10

La console SOC regroupe dix pages fonctionnelles. Le tableau de bord se rafraîchit automatiquement toutes les trente secondes. Le principe fondamental : toute action de protection passe par une file d'approbation. L'analyste doit explicitement valider ou rejeter chaque action avant qu'elle ne s'exécute. Les notifications sont envoyées simultanément sur quatre canaux : le web, l'e-mail, le son et un webhook. Nous vous invitons à constater cela de visu lors de la démonstration.

↪ Examinons d'abord les résultats de nos tests de validation.

Diapo 20/24 — TESTS & RÉSULTATS — 9 :40

Nous avons validé notre système sur cinq scénarios d'attaque, allant de sept à vingt-deux événements. Dans tous les cas, la détection s'est effectuée en moins de cinq secondes, sans perte d'événement. Le pipeline a été validé de bout en bout, de l'événement initial jusqu'à la timeline d'audit. Il convient de noter honnêtement une limite : les tests ont été conduits exclusivement sur Linux. Un attaquant qui ralentirait délibérément son rythme resterait moins visible au moteur.

↪ Passons maintenant à la démonstration.

Diapo 21/24 — DÉMO LIVE — 10 :00, checklist 5 min, hors chrono présentation

1. Ouvrir `http://10.20.0.1` (console SOC)
2. Se connecter : identifiants + **code 2FA**
3. Vérifier : 3 agents en ligne (voyant vert)
4. **Agents** → VM-02 → **Lancer simulation** → « Kill chain complète » (22 évs)
5. **Dashboard** : observer le score monter en temps réel
6. **Alertes** : niveau Critique + signaux détectés
7. **Incidents** : ouvrir l'incident auto-créé, montrer les détails
8. **File d'approbation** : approuver l'isolation réseau
9. **Timeline** : tout est horodaté et tracé

Rester calme. Laisser le temps aux événements d'arriver. Ne pas se précipiter.

↪ **Revenir à la présentation pour conclure.**

Diapo 23/24 — CONCLUSION & PERSPECTIVES — après démo

Excellence Monsieur le Président du jury, Mesdames et Messieurs les membres du jury, permettez-nous de vous présenter la synthèse de ce travail. RansomShield est une solution opérationnelle qui couvre l'ensemble du cycle de traitement d'un incident de sécurité : collecte, analyse, alerte, réponse et audit. Elle repose sur un agent de surveillance multi-plateforme, une console complète, un moteur de détection réactif et un contrôle humain systématique à chaque décision. Ce travail présente des limites que nous assumons pleinement : il a été réalisé dans un environnement contrôlé, avec des réponses partiellement simulées, et testé exclusivement sur Linux. Nous n'avons nullement la prétention d'avoir épuisé le sujet. Les perspectives sont clairement identifiées : intégration du chiffrement HTTPS, mise en place de rôles utilisateurs distincts, puis à terme du machine learning pour affiner la détection, et une intégration avec les outils SIEM du marché. Nous vous remercions pour l'attention que vous avez bien voulu accorder à notre travail. Nous restons à votre entière disposition pour vos questions.

↔ **Sourire, poser les mains, attendre les questions.**

Diapo 24/24 — QUESTIONS DU JURY — aide-mémoire

- **Pourquoi Python et pas C/Go ?** → Watchdog exploite inotify nativement, CPU minimal, portable Linux/Windows/macOS
- **Faux positifs ?** → Seuils configurables ; l'**opérateur humain** valide ou rejette chaque action
- **Pourquoi pas HTTPS ?** → LAN contrôlé ; c'est la 1^{re} perspective pour la production
- **Différence avec un antivirus ?** → Comportement, pas signature ; fonctionne sur variantes inconnues
- **Testé sur Windows ?** → Non, limite assumée ; l'agent est compatible via Watchdog, non testé dans ce travail
- **Passage à l'échelle ?** → Tests sur 3 VM ; l'API Laravel est conçue pour monter en charge, tests de perf à conduire

*Reformuler la question avant de répondre. Garder les **captures d'écran** prêtes.*